

Git Branch Naming and Commit Message Conventions

To ensure consistency, readability, and collaboration efficiency in version control systems, adhering to well-defined Git branch naming and commit message conventions is essential. This document outlines best practices for naming branches and structuring commit messages, complete with examples.

Git Branch Naming Conventions

General Guidelines

- Use lowercase letters.
- Separate words with hyphens (-) for readability.
- Use concise, descriptive names.
- Avoid using special characters or spaces.
- Include relevant identifiers such as issue numbers or task IDs where applicable.

Branch Types

1. Feature Branches
 - For developing new features.
 - Naming convention: feature/<short-description>
 - Example: feature/add-user-authentication
2. Bugfix Branches
 - For fixing bugs.
 - Naming convention: bugfix/<short-description>
 - Example: bugfix/fix-login-error
3. Hotfix Branches
 - For urgent fixes in production.
 - Naming convention: hotfix/<short-description>
 - Example: hotfix/critical-payment-bug
4. Release Branches
 - For preparing a new production release.
 - Naming convention: release/<version>
 - Example: release/1.0.0
5. Support Branches
 - For maintaining legacy versions.
 - Naming convention: support/<version>
 - Example: support/0.9.x
6. Experimental Branches

- For testing experimental features.
- Naming convention: experimental/<short-description>
- Example: experimental/new-ux-design

7. Documentation Branches

- For updating or creating documentation.
- Naming convention: docs/<short-description>
- Example: docs/api-endpoints

Git Commit Message Conventions

Structure of a Commit Message

A well-structured commit message typically consists of:

1. Header

- A single line summarizing the change.
- Maximum 50 characters.
- Written in the imperative mood (e.g., "Add support for dark mode").

2. Body (optional)

- Detailed explanation of what was changed and why.
- Should include context and reasoning behind the change.
- Line length should not exceed 72 characters.

3. Footer (optional)

- Used for referencing issues, tasks, or metadata like breaking changes.

Commit Type Prefixes

Use prefixes to categorize commits:

- feat: – New features.
- fix: – Bug fixes.
- docs: – Documentation updates.
- style: – Code style changes (formatting, no functional changes).
- refactor: – Code restructuring without changing behavior.
- test: – Adding or updating tests.
- chore: – Maintenance tasks (e.g., dependency updates).
- perf: – Performance improvements.
- build: – Changes to build scripts or dependencies.
- ci: – Continuous integration or deployment changes.

Examples of Commit Messages

Simple Commit Message

feat: add user authentication module

Commit Message with Body

fix: resolve login page crash on mobile devices

The issue was caused by incorrect media queries in the CSS. Adjusted the breakpoints and tested on various screen sizes to ensure compatibility.

Fixes: 123

Commit Message with Breaking Changes

refactor: update API response structure

The API response format has been updated to align with RESTful standards. This change is backward-incompatible and requires client updates.

BREAKING CHANGE: `userDetails` object is now nested under `data` key.